Project Report: Multi-Agent Reinforcement Learning for Maze Exploration

Vamsi Krishna [#20298835] Yue Xin Li [#20307667]

April 30 2025

Abstract

Using Multi-Agent Reinforcement Learning (MARL) for better exploration and collaboration in terms of efficiency, scalability is of great interest to a broad range of fields such as disaster response, defence, healthcare, and video games. Using multiple agents over a single agent comes with many advantages: they are more economical, more scalable and less susceptible to overall failure. It also catalyzes many novel applications like nanobot swarms, humanoid locomotion etc. that naturally lend themselves as multi-agent problems. Our focus is the exploration part of the problem where the primary goal is to learn effective exploration strategies by using multiple agents in maze environments. We tackle this by first implementing an independent DQN agent that is able to reach a goal, followed by cooperative agents which use the QMIX algorithm. The QMIX algorithm monotonically combines the Q-values of the agents to ensure any improvement to the combined Q-values results in better individual performance. We find that QMIX shows promising behaviors of improving the cooperation (measured by revisits), exploration (measured by exploration %), and overall faster maze solving (measured by steps to solve).

1 Introduction

Multi-Agent Reinforcement Learning (MARL) is a crucial field for developing autonomous agents that can collaborate, compete and coexist. Our project focuses on the potential of MARL for

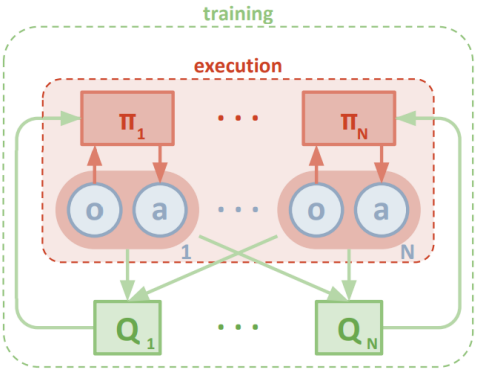

Figure 1: Centralized training with decentralized execution (CTDE), the main idea behind the QMIX algorithm (source: Lowe et al. [2020])

search and rescue operations, specifically on goal-finding and exploration. Our goal is to implement MARL agents that cooperate in a maze-solving context. To do so, we start with a DQN single agent implementation, then expand to a cooperative QMIX algorithm. Using multiple agents over a single agent comes with many advantages: they are more economical, more scalable and less susceptible to overall failure. This can lead to faster exploration and task completion, as they are able to cover more ground simultaneously and explore in parallel, as opposed to only sequentially. Our project will explore the challenges of this method, such as effective exploration (i.e, avoiding redundancy) and collaboration, and navigating in partially observable environments.

MARL has many applications in the context of maze solving: it enables teams of robots to explore unknown environments without centralized control efficiently. Agents can learn to explore large, non-overlapping parts, solve the maze with fewer moves, and optimize navigation by balancing exploration and exploitation, while trying to avoid collisions with obstacles or with each other. For example, in search-and-rescue scenarios, autonomous robots can collaboratively map an area, ensuring full coverage while minimizing redundancy. Similarly, in swarm robotics like multi-drone navigation, MARL allows for adaptive routing to efficiently guide agents toward exits or goals.

Figure 1 shows the CTDE framework based on which the QMIX algorithm is built. The main idea behind QMIX is to learn a global Q-function that then gets monotonically decomposed to individual agents' Q-functions, making sure they all contribute to effective maze exploration while invoking cooperative behaviors in them. This is discussed more in the following sections.

2 Related Work

MARL has become an essential framework for training autonomous systems that must coordinate or compete in shared environments. Early work such as Tan [1993] laid the foundation for cooperative and competitive multi-agent learning. More recent approaches like MADDPG (Multi-Agent Deep Deterministic Policy Gradient) Lowe et al. [2017] introduced centralized training with decentralized execution (CTDE) to address the non-stationarity problem in MARL. MAPPO (Multi-Agent Proximal Policy Optimization) Yu et al. [2021] further stabilized training through an adapted version of PPO for multi-agent settings, achieving strong results across standard MARL benchmarks like StarCraft Multi-Agent Challenge (SMAC).

Collaborative exploration in MARL focuses on agents learning exploration strategies that maximize collective knowledge gain while minimizing redundancy. Value Decomposition Networks (VDNs) Sunehag et al. [2017] offered an earlier approach to factorizing joint value functions in cooperative MARL by representing the global Q-value as a sum of individual agent Q-values. QMIX Rashid et al. [2018] proposed a monotonic value function factorization to enable scalable cooperative learning, which has been particularly effective in structured tasks like exploration. Furthermore, works such as MAVEN (Multi-Agent Variational Exploration) Mahajan et al. [2019] emphasized the importance of structured exploration through latent variable conditioning to promote diverse behavior among agents. Studies like Jaques et al. [2019] introduced social influence as an intrinsic motivation mechanism to improve coordination during exploration. In robotics, approaches such as Huttenrauch et al. [2019] demonstrated how decentralized policies learned through MARL can enable scalable, robust multi-robot exploration. Our project builds upon these methods by implementing QMIX-based cooperation strategies to tackle maze-solving with an affinity for exploration.

3 Background and Frameworks

We model the multi-agent environment as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP), which is defined by a tuple $(\mathcal{S}, \mathcal{U}, P, r, \mathcal{O}, Z, n, \gamma)$ as seen in Rashid et al. [2018] where:

- $\mathcal{S}$ is the set of environment states.
- $\mathcal{U}$ is the set of actions available to each agent; the joint action space is $\mathcal{U}^n$ for n agents.
- $P : \mathcal{S} \times \mathcal{U}^n \times \mathcal{S} \to [0, 1]$ is the state transition function, defining the probability of transitioning to a new state s' given current state s and joint action $\mathbf{u} \in \mathcal{U}^n$.
- $r : \mathcal{S} \times \mathcal{U}^n \to \mathbb{R}$ is the reward function shared by all agents, mapping state and joint action to a scalar reward.
- $\mathcal{O}$ is the set of possible observations for each agent; the joint observation space is $\mathcal{O}^n$.
- $Z : \mathcal{S} \times \mathcal{O}^n \to [0, 1]$ is the observation probability function, giving the probability of joint observation $\mathbf{o} \in \mathcal{O}^n$ given state s.
- n is the number of agents.
- $\gamma \in [0, 1]$ is the discount factor.

At each time step, the environment is in some state $s \in \mathcal{S}$. Each agent $a \in \{1, \ldots, n\}$ receives a private observation $o_a \in \mathcal{O}$ according to the observation function $Z(s)$. Based on its own observation o_a, each agent selects an action $u_a \in \mathcal{U}$. Together, the agents' actions form a joint action $\mathbf{u} = (u_1, \ldots, u_n) \in \mathcal{U}^n$, which causes the environment to transition to a new state $s' \sim P(\cdot \mid s, \mathbf{u})$ and generates a shared reward $r(s, \mathbf{u})$.

Each agent learns a stochastic policy $\pi_a(u_a \mid \tau_a)$, where τ_a is the action-observation history of agent a, and $\pi = (\pi_1, \ldots, \pi_n)$ denotes the joint policy across all agents. The goal in the Dec-POMDP framework is to maximize the expected discounted return.

We adapted a gym-compatible Maze environment to support multiple agents and goals. There are two action modes - one is without diagonal moves that just supports left, right, up, down moves in the maze; the other supports diagonal moves (top left, top right, bottom left, bottom right). We're using a randomly generated block maze with 10% obstacles which means 10% of the total cells are walls. Here's a link to the repo: https://github.com/rpinsler/gym-maze/tree/master

We used *skrl* Serrano-Muñoz et al. [2023], which implements a few single-agent and multi-agent reinforcement learning algorithms. It also has built-in support for many environments for both the single- and multi-agent scenarios. We used DQN Agent which has already been implemented while the training/eval loop and plotting logic is added. We implemented the QMIX algorithm from scratch by extending multi-agent base class. Here's a link to the repo: https://github.com/Toni-SM/skrl/

4 Project Method

The idea is that, based on observations, the agents develop strategies to efficiently explore the maze and solve it in the shortest possible time by minimizing the number of steps. Here are the equations for reward function, transition function, actions:

$$R(s,a,s') = \begin{cases} +1 \cdot |\text{maze}| & \text{if goal\_reached} \\ -1 & \text{if } s' = s \quad \text{(hit a wall)} \\ -0.01 \cdot |\mathcal{A}| + 0.5 & \text{if first\_visit} \\ -0.01 \cdot |\mathcal{A}| - 0.2 \cdot \min(\text{revisit\_count}, 5) & \text{otherwise} \end{cases} \tag{1}$$

where

- $R(s,a,s')$: Reward for transitioning from state s to s' via action a
- $|\text{maze}|$: Size of the maze
- $|\mathcal{A}|$: Number of possible actions

$$p(s_{i+1}|s_i,a_i) = \begin{cases} s_i, \text{if the next state is a wall} \\ s_{i+1}, \text{otherwise} \end{cases} \tag{2}$$

$$a_i = \begin{cases} \text{take a step towards left, if } a_i = 0 \\ \text{take a step towards right, if } a_i = 1 \\ \text{take a step downward, if } a_i = 2 \\ \text{take a step upward, if } a_i = 3 \\ \text{take a step towards top left diagonally, if } a_i = 4 \\ \text{take a step towards top right diagonally, if } a_i = 5 \\ \text{take a step towards bottom left diagonally, if } a_i = 6 \\ \text{take a step towards bottom right diagonally, if } a_i = 7 \end{cases} \tag{3}$$

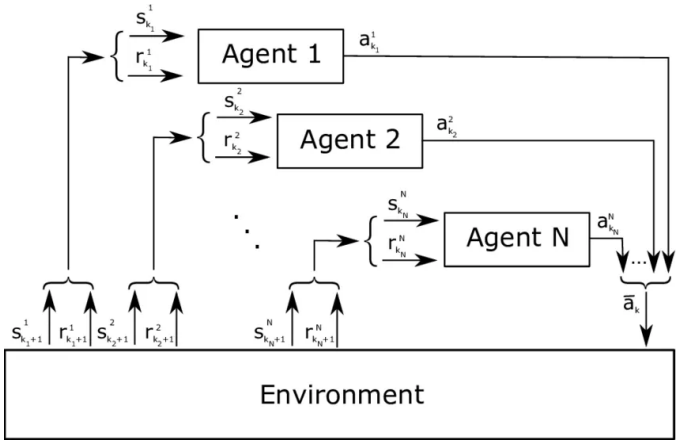

Figure 2: Multi-Agent interaction with the Environment

Reward Function: Eq (1) penalizes for taking steps but that penalty is outweighed by the maze-solved reward incentivizing the agents to take steps despite the penalties. The equation defines the reward function for a state s. Let's break down the reward function - big reward if the goal is reached, a penalty of -1 if run into a wall, a small penalty of -0.01 for every move, a 0.5 reward for visiting a new state, an increasing penalty for revisits capped at -1. The overall objective is to maximize the sum of rewards across agents and the total reward for each agent is the discounted

sum of rewards it receives, where γ is the discount factor, $r_i(t)$ is the reward of agent i at time step t, and N and T represent the number of agents and total time steps, respectively:

$$\text{Maximize} \quad \sum_{i=1}^{N} \sum_{t=0}^{T} \gamma^t r_i(t) \tag{4}$$

State and Action Spaces: The state space for each agent is the blocks in the maze that are 1 unit apart (so in most cases, the agent has 8 blocks - 4 diagonally, 2 vertically, 2 horizontally) so the dimensionality is 8, and the action space can just be an integer number in the range [1, 8] interpreted as taking a step in some direction as laid out in Eq (3). **State Transitions**: As shown in Eq (2) the transitions are deterministic i.e, the agent ends up in the block to where it took the action to move unless there's a wall (if there's a wall the agent stays in the same location). As shown in Figure 2, the agents take actions based on their partial observations of the state. Concretely, the way agents interact with the environment is the following - observation gathering, taking the actions based on individual policies, collect reward sequentially. Observations are gathered for all of the agents, they are passed to their respective agents and their individual actions are all collected, then these actions are applied on the environment sequentially and the positions of the agents are updated, and the environment returns the combined reward, the new positions of the agents, and their observations.

To address the challenges of effective exploration in multi-agent settings, we began with a Deep Q-Network (DQN) as our baseline. DQN is well-suited for environments with discrete action spaces and has proven performance in single-agent scenarios. It allowed us to first understand the task complexity and tune the reward structures within the maze environment. This agent, trained independently, helped establish the lower bound performance for maze solving, exploration efficiency, and overall step minimization. Using DQN helped confirm the feasibility of reaching the goal and analyzing individual exploratory behaviors.

However, to tackle the full complexity of cooperative multi-agent exploration, we needed a method capable of learning coordinated strategies. This led us to implement the QMIX algorithm. Unlike DQN, which learns a single-agent Q-function, QMIX allows for cooperative behavior among agents by combining individual agent Q-values into a global Q-function in a monotonic way. This ensures that optimizing the global Q-function implicitly leads to better individual agent decisions.

QMIX fits our project goals in several key ways: CTDE is ideal for scenarios like distributed search and rescue where communication during execution is limited but training can be centralized; agents are encouraged to explore non-overlapping areas of the maze due to a shared reward structure and the revisit penalty embedded in the reward function. DQN served as a foundational agent to validate the environment and reward design, while QMIX enabled emergent cooperative behavior aligned with the project objective of improving exploration, goal-seeking through MARL.

Deep Q-Network: DQN is a value-based reinforcement learning algorithm that approximates the Q-function using a deep neural network. It uses experience replay and target networks to stabilize training. The agent observes a state, selects an action using an ϵ-greedy strategy, and updates its Q-network using the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{5}$$

Where s is the current state, a is the action taken, r is the reward received, s' is the next state, and α is the learning rate.

In our project, DQN was used in a single-agent mode to establish a baseline for maze solving. It allowed the agent to learn optimal paths and strategies in isolation. The learned Q-function

provided insight into which regions of the maze were more frequently visited and helped evaluate exploration effectiveness before extending to multi-agent coordination.

QMIX: The crux of the QMIX algorithm lies in its approach to multi-agent reinforcement learning that balances centralized training with decentralized execution. QMIX employs a mixing network that estimates joint Q-values as a monotonic combination of per-agent values, which are based only on local observations. This monotonicity constraint ensures consistency between centralized and decentralized policies, allowing for tractable maximization of the joint Q-value in off-policy learning. By structurally enforcing that the joint action-value is monotonic in the per-agent values through non-negative weights in the mixing network, QMIX guarantees that the global argmax of the joint action-value function yields the same result as individual argmax operations on each agent's Q-values. This clever factorization enables QMIX to represent a richer class of Q-value functions compared to simpler additive decompositions, while still maintaining the ability to extract decentralized policies that are consistent with the centralized policy. The original architecture from QMIX is given in Figure 3. We used the DQN agents from above as the individual agents and have a mixing network on top of them.

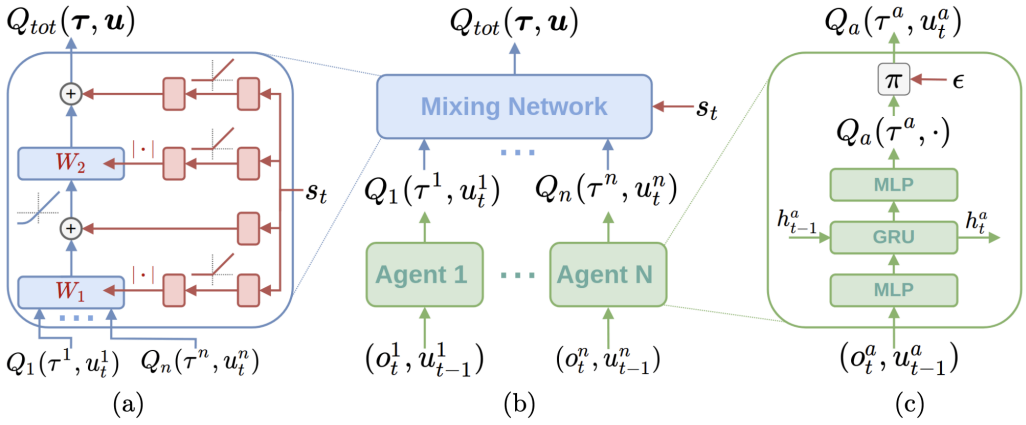

Figure 3: QMIX Architecture: (a) Mixing network structure. In red are the hypernetworks that produce the weights and biases for mixing network layers shown in blue. (b) The overall QMIX architecture. (c) Illustrative agent structure, we used DQN agents.

In this project, we used the Random Block Maze, which we randomly generated with a given maze size and obstacle ratio. Since the pathway was not completely surrounded by walls, the agent could move more freely and explore different ways of navigating the maze. The Q-network for the individual agents is a sequential [Linear, ReLU, Linear, ReLU, Linear] network where the hidden dim is 128. The mixing network is implemented according to the architecture diagram (ref Fig 3) where we have 2 layers of weights and the states are added element-wise after every layer, ELU activation is used after layer 1 as specified in the diagram.

5 Skills Gained

We will acquire a deeper understanding of multi-agent reinforcement learning concepts, more specifically how they share knowledge, how agents work together and how to reward the agents appropriately for the task. We will explore different exploration and pathfinding algorithms and examine

the difficulties in implementing each method. We were able to find the right evaluation metric that would highlight the desired behaviours of the agents.

The technical skills we acquired include hyperparameter tuning, choosing and modifying environments to evaluate the agents' behaviour. We also had to adapt those existing environments to meet requirements for multi-agents. We worked on visualizations of the maze, and plotting the results using Tensorboard. For training and testing, we had to finding the metrics to track and evaluate, manage how to store the data by populating the replay buffer. Finally, we had to tune the reward function, which was originally intended for single agents, for multi agents to better reflect their performance. All in all, this allowed us to explore understand existing DQN and QMIX approaches, and compare our results to those obtained in the original papers. We were able to identify the challenges of these methods by adapting them to a different context.

6 Experiments and Analysis

We came up with 3 helpful metrics, namely maze exploration percentage, steps to solve, and number of revisits, as a proxy for exploration, goal-seeking, and collaborative behaviors, respectively. The mazes are randomly generated with size 15×15 and 10% obstacles (i.e, 10% of the cells are walls). We ran extensive experiments to tune the DQN hyperparameters because it's key to get DQN working as it's used for the agents in the QMIX algorithm. The values considered for learning rate, batch_size, observability, gamma are (1e-4, 3e-4, 1e-3, 3e-3), (64, 256, 1024), (full, partial_3, partial_5, partial_7), (0.9, 0.95, 0.99) respectively, and found 1e-3, 64, partial_3, 0.95 to be a good set of parameters. We ran DQN for 150000 steps and a random policy was used for the first 20000 steps, a linearly decaying exploration noise was used.

We then moved on to the experiments with QMIX where we spent quite a bit of time tuning the reward function to observe the desired behaviors of exploration, collaboration, and goal-seeking. It was particularly difficult to get the individual agents to train properly from the gradient signal flowing from the mixing network. For this, we had to ensure the mixing network update frequency is close to the update frequency of the individual agents' Q-networks, and only after this did we see some learning happening. QMIX was taking 5-6x longer than DQN and we ran the experiments for 50000 steps and a random policy was used for the first 2000 steps, a linearly decaying exploration noise was used. We tried different update frequencies (100, 500, 1000, 1500) for the networks and found 500, 1000 to be good values possibly because each episode is roughly 500 steps and this ensured we had seen at least one full episode before updating the networks. Tuning the DQN beforehand was a big plus here because it would have been really difficult to tune everything at once in QMIX.

We see DQN performance in Fig. 4 where the rewards are slightly improving, loss is reducing while the steps to solve is going down along with revisits. Revisits is always less than steps to solve, so steps to solve should be a good indicator of the goal-seeking behavior. Also, steps to solve only includes those mazes that were solved. Some random mazes that had walls around a goal which made it impossible to reach which would explain revisits being higher, in some instances, than the step size. At this point, it was clear that we could move on to QMIX experiments.

We have QMIX results in Fig. 5 where the rewards trend higher, loss trends lower. And the maze completion metrics show an improving trend where steps to solve and revisits are going down while the exploration is also observed. We had to make another change to the reward function at this point to avoid solving unsolvable mazes - after 1500 steps in the maze, the episode would stop with a penalty of -500. This ensured that "bad" training examples were limited and made the training faster while also improving the metrics. We include the evaluation performance of QMIX

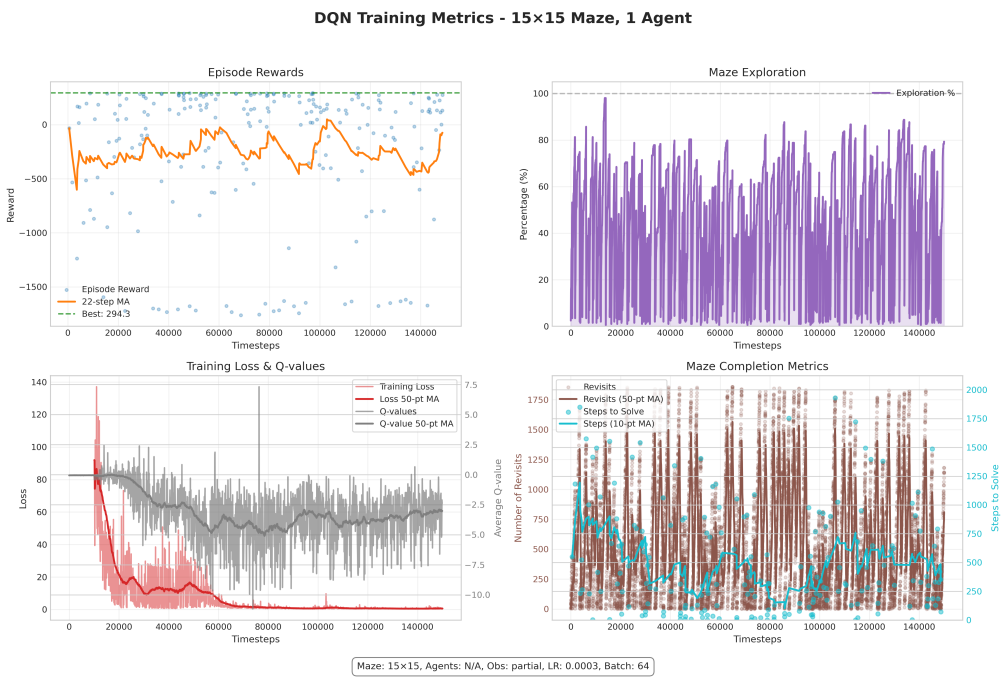

Figure 4: (i) The rewards are generally improving as evidenced by fewer points in [-500, -1500] in the second half of the training (moving average doesn't quite tell the whole story), (ii) seeing some good exploration percentages suggesting that the policy learns the desired behaviour, (iii) loss consistently goes down while the Q values show a slight upward trend, (iv) steps to solve goes down consistently to about an average of 260, revisits average to about 250

after solving 150 mazes in Table 1. It's clear from the results that QMIX performs well and the desired behaviors of collaboration, exploration, goal-seeking are observed via revisits, exploration %, steps to completion metrics respectively.

Table 1: QMIX Performance Summary Across 150 Evaluation Episodes

Metric	**Highest**	**Lowest**	**Mean ± Std. Dev.**
Reward	298.16	-998.33	129.31 ± 288.20
Exploration (%)	94.62	3.5	34.27 ± 24.60
Steps to Completion	1265	8	154.91 ± 190.55
Revisits	1085	0	132.39 ± 167.23

7 Video Results

Link to the video results

8 Conclusions

Our DQN algorithm confirms that the agent is able to navigate around obstacles, and learn to reach a goal given partial observability. Our results on QMIX indicate that it achieves the desired results

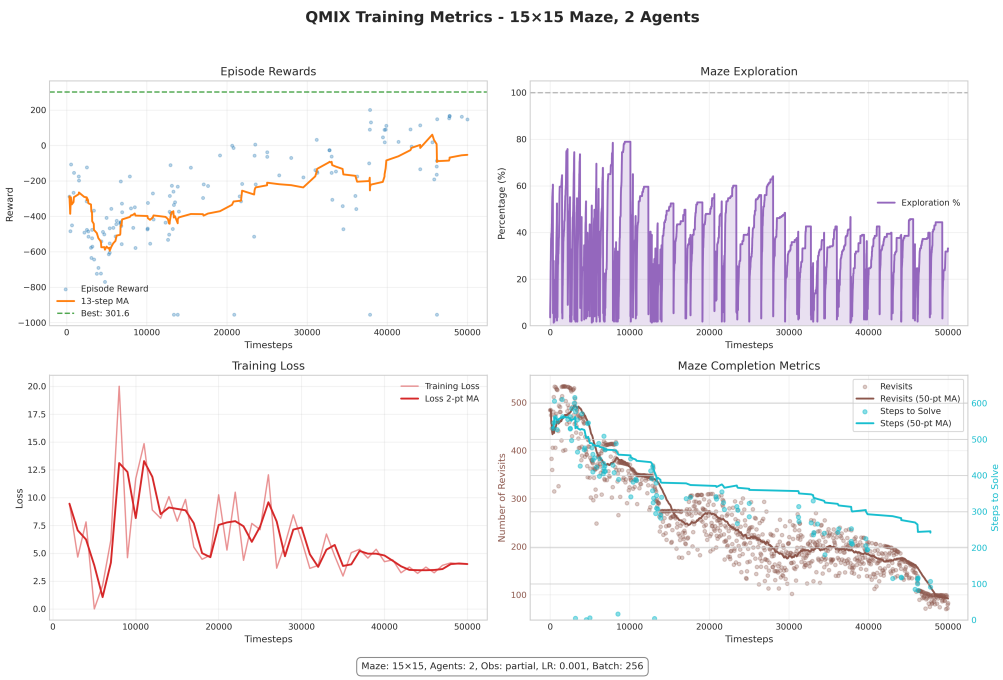

Figure 5: (i) the rewards are improving and stay positive for the most part, (ii) good regions of exploration seen which averages out to 45%, (iii) loss shows a downward trend, (iv) steps-to-solve goes down to about 150 and revisits down to 130

given our objective of exploring the maze and solving the maze in the least number of steps. The algorithm strikes a good balance between the two, which would be useful in a search and rescue operation where resources and time could be limited, and where exploration and coordination is needed since the number of goals is not necessarily equal to the number of agents.

If we had to do things differently, we would have designed better mazes that encouraged cooperation and exploration by using maze generating algorithms. We could have tried to explore different maze solving algorithms, such as ones that could be adapted to continuous environments like StarCraft. We would also probably spend more time on QMIX hyperparameter tuning. This turned out to be 5-6x slower than DQN, and more training could have improved our final performance.

We found that the initial scope of our project was a bit too vague, and that we should have restricted it after experimenting with the environment. Addressing too many aspects proved itself to be extremely challenges, and it would have been more effective to narrow the scope early on and concentrate on a smaller subset of challenges. There were many options we could have explored, from the way the maze is generated, to the obstacles it contained, all the way to the algorithms we used and the way the agents cooperated. These remain ideas that could be explored in future project; using multi objective RL, having negative rewards for dangerous zones, and continuous environments like point mass.

In conclusion, this project got us a glimpse into the implementation of multi-agents and multi-agent objectives, and of all the challenges that come with implementing environments and algorithms adapted to multi-agent goal-seeking and exploration. We are overall satisfied from the journey of completing the project, and find that we have acquired good knowledge about the subjects of MARL.

9 [2 pts] How was the work divided?

Vamsi: Worked on the implementation of DQN, QMIX, plotting functions, ran the experiments, reward function tuning

Yue Xin Set up the gym environment, implemented the simple algorithms (A*, BFS, DFS, Dijkstra), modified the environment for independent multi-agents, prepared the slides for the presentation

10 [16 pts] Code Submission

You will also submit your code for the project. We will have independent grading sessions where students will be asked how parts of the code work. You should also remove extrainiuse file that are generated. For example, run

```
find . | grep -E "(/__pycache__$|\.pyc$|\.pyo$)" | xargs rm -rf
```

You will also submit a *diff* of your code to show what you have added compared to the code you started with for your project.

As an example, the unzipped version of your submission should result in the following file structure. **Make sure that the submit.zip file is below 15MB and that they include the prefix `main_`, `diff_`.**

```
submit.zip
├── data
│   ├── exp1_...
│   │   └── events.out.tfevents.1567529456.e3a096ac8ff4
│   ├── exp2_...
│   │   └── events.out.tfevents.1567529456.e3a096ac8ff4
│   └── ...
├── ift6131_project
│   ├── agents
│   │   ├── ac_agent.py
│   │   └── ...
│   ├── policies
│   │   └── ...
│   └── ...
├── ift6131_project_diff
│   ├── agents
│   │   ├── ac_agent.py
│   │   └── ...
│   ├── policies
│   │   └── ...
│   └── ...
├── README.md
└── ...
```

This example is based on your course assignments. It is okay if the file names for your experiments or classes are different. **What is important is that you include (1) your code, (2) data from your experiments for us to review (3) a *diff* of your code so we can see what you changed.**

11 [10 pts] Presentation Submission

You will also submit a copy of your presentation for grading. This will be submitted before the final presentation is given to the class on April 29th.

12 [5 pts] Peer Review

You will review one of the projects from your other students. This format will closely follow the paper summaries that you have been using for class. You will be graded on how fair and constructive your feedback is to other students.

References

Matthias Huttenrauch, Alexandros Sosic, and Gerhard Neumann. Deep reinforcement learning for swarm systems. *Journal of Machine Learning Research*, 2019.

Natasha Jaques, Angeliki Lazaridou, Edward Hughes, Caglar Gulcehre, Pedro Ortega, DJ Strouse, Joel Z Leibo, and Nando de Freitas. Social influence as intrinsic motivation for multi-agent deep reinforcement learning. In *International Conference on Machine Learning*, pages 3040–3049, 2019.

Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.

Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments, 2020. URL https://arxiv.org/abs/1706.02275.

Anuj Mahajan, Tabish Rashid, Mikayel Samvelyan, and Shimon Whiteson. Maven: Multi-agent variational exploration. In *Advances in Neural Information Processing Systems*, 2019.

Tabish Rashid, Mikayel Samvelyan, Casper de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *International Conference on Machine Learning*, pages 4295–4304, 2018.

Antonio Serrano-Muñoz, Dimitrios Chrysostomou, Simon Bøgh, and Nestor Arana-Arexolaleiba. skrl: Modular and flexible library for reinforcement learning. *Journal of Machine Learning Research*, 24(254):1–9, 2023. URL http://jmlr.org/papers/v24/23-0112.html.

Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z Leibo, Karl Tuyls, et al. Value-decomposition networks for cooperative multi-agent learning. *arXiv preprint arXiv:1706.05296*, 2017.

Ming Tan. Multi-agent reinforcement learning: Independent vs. cooperative agents. *Proceedings of the Tenth International Conference on Machine Learning*, pages 330–337, 1993.

Cheng Yu, Wojciech M Czarnecki, Shaobo Zhang, Da Huang, Abhishek Das, Bilal Piot, Angeliki Lazaridou, Rob Fergus, and Christopher Clark. The surprising effectiveness of ppo in cooperative multi-agent games. In *International Conference on Learning Representations*, 2021.